

Task 1: Project Completion and Final Implementation

- Complete the software solution or quality improvement prototype proposed in Assessment 1.
- Implement the key functional requirements and acceptance criteria.
- Document and justify any changes to scope, requirements, architecture, technology, or implementation.
- Demonstrate the final system through a working prototype.
- Provide a GUI where user interaction is required, or justify an alternative interface such as an API, command-line tool, backend service, automated testing framework, or QA dashboard.

Task 2: Requirements Verification and Traceability

- Review and finalise the functional and non-functional requirements from Assessment 1.
- Update the Requirements Traceability Matrix.
- Map requirements to implemented features, test cases, test results, and defects where applicable.
- Identify any requirements that are partially implemented, unverified, or out of scope, and justify them.

Task 3: Test Execution and Final Test Results

- Execute the planned tests against the completed system.
- Include relevant functional and non-functional tests, such as unit, integration, system, acceptance, regression, usability, security, performance, or accessibility testing.
- Include positive, negative, boundary, invalid-input, and error-handling scenarios where applicable.
- Provide at least *fifteen (15)* final test cases with execution results.
- Summarise the number of planned, executed, passed, failed, blocked, and not-run tests.
- Explain failed tests, corrective actions, regression testing, and remaining quality risks.

Task 4: Test Automation and Quality Engineering

- Identify suitable test cases for automation and justify the selection.
- Implement automated tests using an appropriate tool or framework where suitable.

- Report automated test execution results using logs, screenshots, reports, or repository evidence.
- Discuss the benefits, limitations, and maintainability of the automated test suite.
- Where automation is not appropriate, justify the use of manual or exploratory testing.

Task 5: Continuous Integration and DevOps QA

- Describe the development and integration workflow.
- Implement a simple CI workflow or equivalent automated quality check where appropriate.
- Define quality gates such as successful build, passing tests, linting, code review, or coverage threshold.
- Demonstrate smoke or sanity testing where relevant.
- If CI/CD is not technically appropriate, justify an alternative quality workflow.

Task 6: Defect Management and Root Cause Analysis

- Maintain a final defect register covering defects found during development and testing.
- Define severity, priority, and defect status categories.
- Describe the defect lifecycle used in the project.
- Analyse at *least three (3)* significant defects using root cause analysis.
- Propose corrective actions and defect-prevention measures.
- Explain how defect analysis improved product or process quality.

Task 7: Quality Testing

- Conduct quality testing relevant to the project, such as security, performance, usability, accessibility, compatibility, reliability, or maintainability testing.
- Address at *least two (2)* quality areas where applicable.
- Use risk-based reasoning to justify the selected testing activities.
- Document testing evidence, results, limitations, and recommended improvements.

Task 8: Quality Metrics and Final Release Decision

- Present relevant quality metrics, such as test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, or commit activity.
- Evaluate the completed system against requirements, acceptance criteria, and quality attributes.

- Summarise testing results, defect status, metrics, limitations, and residual risks.
- State whether the system is ready for release, conditionally ready, or not ready for release.
- Justify the release decision using evidence.

Task 9: AI-Assisted Development and Critical Reflection

- Explain how GitHub Copilot was used for implementation, testing, debugging, refactoring, or QA activities.
- Provide representative evidence of AI-assisted development.
- Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected.
- Identify any quality risks found in AI-assisted outputs, such as incorrect logic, insecure code, poor maintainability, or unsuitable assumptions.
- Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice.

Task 10: Quality Improvement and Future Recommendations

- Identify the main quality lessons learned from the project.
- Recommend improvements to the development and QA process.
- Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability.
- Explain how the project could evolve toward stronger continuous quality engineering practice.

Task 11: Team Contribution and Professional Reflection

- Provide a contribution statement for each group member.
- Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks.
- Include a brief individual reflection from each group member.

Task 12: Project Presentation and Demonstration

Each group must prepare a 15-slide presentation and demonstrate the completed project. The presentation should clearly summarise the problem, final solution, quality assurance activities, testing evidence, and final quality evaluation. The presentation should cover:

- Project title, group members, problem, stakeholders, and scope
- Key functional and non-functional requirements
- Final prototype, implemented features, and changes after Assessment 1
- Testing approach, coverage, execution results, and key findings
- Automation, CI/CD, or equivalent quality workflow
- Defects, root cause analysis, and corrective actions
- Quality metrics, advanced testing, and release decision
- GitHub Copilot use, limitations, risks, and future improvements
- Individual contributions and lessons learned.

All students must participate in the demonstration and show the working prototype or quality improvement solution. The demo should include key features, important test cases, automated tests or CI results where applicable, defect fixes, and evidence that major requirements have been satisfied. Students must be prepared to answer questions about the project, code, testing, quality artefacts, AI-assisted development, and individual contributions.

Deliverables

- Final Project and Software Quality Assurance Report: *6000--7000 words*
- Source code and GitHub repository link; provide evidence from GitHub commits.
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report